CAP 5610 Introduction to Machine Learning: Midterm Report

Ansu Priyadarshi · Panther ID 6617174 · Dr. Adigun · 19 October 2026

1. Logistic Regression: Adult Census Income

The Adult Census task was modeled with two logistic-regression pipelines using identical preprocessing. "?" entries in `workclass`, `occupation` and `native_country` were treated as missing; numerical predictors (`age`, `education_num`, `hours_per_week`) were median-imputed and standardized, and the remaining predictors, including `capital_gain` and `capital_loss` as specified in the handout, were mode-imputed and one-hot encoded (304 inputs in total). Both models used $C = 1.0$, `max_iter=1000` and `random_state=42`. The required model used `class_weight="balanced"`; the comparison model omitted class weighting. A majority-class predictor (always "$\leq$\$50K") was the naive baseline (Table A1).

The baseline accuracy was 75.88% (F1 = 0, ROC-AUC = 0.5). The balanced model achieved 82.48% accuracy, F1 = 0.7015 and ROC-AUC = 0.9226; the unweighted model achieved 86.95% accuracy, F1 = 0.6996 and ROC-AUC = 0.9219. Both models therefore learned well beyond the baseline, and the two are essentially tied on F1 and ROC-AUC: weighting shifted the operating point rather than the ranking ability. The confusion matrices (Table A2) show the trade-off. The balanced model raised minority-class recall from 0.6302 to 0.8536 and correctly identified 351 additional high-income cases, but produced 642 more false positives (911 vs. 269), so precision fell from 0.7863 to 0.5955 and total errors rose from 850 to 1,141, which is why its accuracy is lower. Because the assignment emphasizes the minority class, the balanced model is the primary model, although the unweighted model has the higher raw accuracy. The coefficient ranking was almost unchanged without weighting (Table A3).

For logistic regression, $\log \frac{p}{1-p} = \beta_0 + \sum_j \beta_j x_j$, so a positive β increases the log-odds of income above \$50K and a negative β decreases them, holding other inputs fixed; e^{β} is the multiplicative change in the odds. The largest balanced-model coefficient was `capital_gain_99999` ($\beta = 4.459$) and the most negative was `capital_gain_5013` ($\beta = -3.644$). These exact-dollar categories are sparse, so their extreme weights should be read cautiously. Among the other predictors, `education_num` ($\beta = 0.748$) gives $e^{0.748} \approx 2.11$ times the odds per one-standard-deviation increase. Because every one-hot level is kept, levels of the same variable are compared by their difference: Married-civ-spouse ($+1.488$) vs. Never-married (-1.153) gives $e^{2.641} \approx 14$ times the odds (Table A3).

2. Neural Network Regressor: King County Housing

Neither King County file contained missing values; imputers were kept in the pipeline for robustness. The 17 numerical predictors were standardized, `zipcode` was one-hot encoded because it is a location label rather than a quantity (87 inputs), and `log1p` was applied to the right-skewed price target. 20% of the training file was held out for validation; the test set was used only once, for the final evaluation. Sigmoid and ReLU `MLPRegressor` models used the same settings: hidden layers (64, 32), Adam, `learning_rate_init=0.001`, `max_iter=500`, early stopping (`validation_fraction=0.10`, `n_iter_no_change=20`) and `random_state=42`, so the hidden-layer activation was the only difference (Table A4, Figure A1).

ReLU converged faster (169 iterations, 26.42 s) than sigmoid (350 iterations, 55.91 s), but sigmoid predicted much more reliably. On the validation log-price scale, R^2 was 0.8972 for sigmoid versus 0.8026 for ReLU. After inverse transformation ReLU produced an extreme \$1.327 billion prediction for a \$640,000 house, driving its dollar-scale R^2 to -3579.23, while sigmoid kept $R^2 = 0.8798$. The cause was one validation house recorded with 33 bedrooms, about 33 standard deviations above the mean and far outside the training range (maximum 10). ReLU is linear for positive inputs, so it extrapolated this input without bound (log-price 21.0), whereas the sigmoid saturates and stays bounded.

The final sigmoid network was refit on all 16,613 training houses and stopped after 285 iterations. On the test set it achieved MSE $= 1.421 \times 10^{10}$ (RMSE $\approx$ \$119,200) and $R^2 = 0.9019$, compared with MSE $= 1.449 \times 10^{11}$ (RMSE $\approx$ \$380,700) and $R^2 = -0.0005$ for the baseline that predicts the training-mean price (\$538,144) for every house (Table A5). The network cut the squared error roughly tenfold and explained about 90% of the variance in test prices. Faster convergence did not imply better generalization, and the inverse exponentiation magnified extreme log-scale errors.

3. Neural Network Classifier: CIFAR-10

Images were converted to floating-point values in $[0, 1]$ (`ToTensor`), flattened from $3 \times 32 \times 32$ to 3,072 inputs, and the 50,000 training images were split into 45,000 training and 5,000 validation images; the official 10,000-image test set was reserved for the final evaluation. Fully connected ReLU networks were trained with Adam (learning rate 0.001), cross-entropy loss, batch size 256 and 15 epochs. Four architectures compared shallow/narrow [128], shallow/wide

[512], medium [256, 128] and deep/narrow [128]×4 networks (Table A6, Figure A2). The [256, 128] network was best, reaching 51.32% validation accuracy at epoch 14 with 820,874 parameters. Neither the widest network (1,578,506 parameters, 49.10%) nor the four-layer narrow network (49.56%) improved validation accuracy. Training time grew with capacity: the wide network was slowest (246 s vs. 150 s for [128]), and the four-layer network took longer than the one-layer [128] network despite only 13% more parameters, because its layers are computed sequentially. The tested range stayed within the limits of 8 hidden layers and 1,000 neurons.

Dropout was then applied after each hidden layer of the selected architecture. With train and validation losses both measured in evaluation mode (dropout off), the generalization gap fell from 0.1455 without dropout to 0.0998 at dropout 0.10 and 0.0567 at dropout 0.30, so the regularizer did reduce the gap. However, best validation accuracy declined from 51.32% to 49.70% and 46.06%, showing increasing underfitting (Table A7, Figure A3). After 15 epochs the unregularized model was only mildly overfitting, so dropout mainly removed useful capacity; it would be expected to help more for larger networks trained longer. The unregularized network was therefore retained, retrained from scratch on all 50,000 training images for 14 epochs (its best validation epoch), and obtained 50.75% test accuracy with test cross-entropy 1.3969 (Table A8). The confusion matrix (Figure A4, Table A9) shows the largest directed error was cat→dog (336 images); cat (29.7%), deer (32.2%) and bird (39.2%) were the hardest classes, and automobile↔truck (179/150) and airplane→ship (148) were the other frequent confusions, consistent with flattened pixels losing the spatial structure that separates visually similar classes. Overall, moderate width and depth gave the strongest trade-off; more capacity or more regularization did not automatically improve generalization.

Overall observations

Across the three tasks, a model should be judged by the metric that matches the problem rather than by complexity or speed alone. Class weighting in Task 1 changed the precision–recall trade-off with almost no change in ROC-AUC; ReLU in Task 2 converged faster but extrapolated unstably on an extreme input; and greater width, depth or dropout in Task 3 did not improve validation accuracy. The final selections therefore favored reliable held-out performance and task-appropriate error behavior over raw capacity or training speed.

Appendix

A. Task 1: Adult Census logistic regression

Table A1: Adult Census test-set performance (6,513 people).

Model	Accuracy	F1	ROC-AUC	Precision	Recall
Majority-class baseline	0.7588	0.0000	0.5000	–	0.0000
Balanced logistic regression	0.8248	**0.7015**	**0.9226**	0.5955	**0.8536**
Unweighted logistic regression	**0.8695**	0.6996	0.9219	**0.7863**	0.6302

Settings: $C = 1.0$, `max_iter=1000`, `random_state=42`; the balanced model also used `class_weight="balanced"`.

Table A2: Test-set confusion matrices (rows = true class, columns = predicted class).

	Balanced		Unweighted	
	pred. $\leq$50K	pred. $>$50K	pred. $\leq$50K	pred. $>$50K
true $\leq$50K (4,942)	4,031 (TN)	911 (FP)	4,673 (TN)	269 (FP)
true $>$50K (1,571)	230 (FN)	1,341 (TP)	581 (FN)	990 (TP)
total errors	1,141		850	

Table A3: Largest coefficients (β, log-odds of income $>$\$50K).

Feature / level	Balanced β	Unweighted β	Comment
`capital_gain_99999`	$+4.459$	$+4.189$	largest positive overall
`capital_gain_8614`	$+4.348$	$+4.259$	large positive capital-gain level
`capital_gain_14344`	$+4.158$	$+3.853$	large positive capital-gain level
`capital_loss_1564`	$+3.912$	$+3.801$	large positive capital-loss level
`marital_status_Married-civ-spouse`	$+1.488$	–	$e^{1.488-(-1.153)} \approx 14\times$ odds of Never-married
`education_num` (standardized)	$+0.748$	–	$e^{0.748} \approx 2.11\times$ odds per 1 SD
`marital_status_Never-married`	-1.153	–	lower odds than married levels
`occupation_Priv-house-serv`	-1.471	–	most negative non-capital level
`capital_loss_1740`	-3.078	-2.724	large negative capital-loss level
`capital_gain_4064`	-3.213	-2.930	large negative capital-gain level
`capital_gain_5013`	-3.644	-3.282	most negative overall

Positive coefficients increase the log-odds of income above \$50K, negative ones decrease them, holding other inputs fixed. Exact capital-value categories are sparse and dominate the extreme coefficients. "–" = not in the unweighted model's printed top-10 lists.

B. Task 2: King County neural-network regressor

Table A4: Controlled sigmoid vs. ReLU comparison on the validation split (3,323 houses).

Activation	Iterations	Time (s)	Log MSE	Log R^2	Dollar MSE	Dollar R^2
Sigmoid	350	55.91	**0.028885**	**0.8972**	$\mathbf{1.779 \times 10^{10}}$	**0.8798**
ReLU	**169**	**26.42**	0.055486	0.8026	5.298×10^{14}	-3579.23

Both: hidden layers $(64, 32)$, Adam, `learning_rate_init=0.001`, `max_iter=500`, `early_stopping=True`, `validation_fraction=0.10`, `n_iter_no_change=20`, `random_state=42`. Dollar MSE is in dollars squared. ReLU's largest error: a \$640,000 house recorded with 33 bedrooms was predicted at \$1.327 billion (sigmoid: \$326,695).

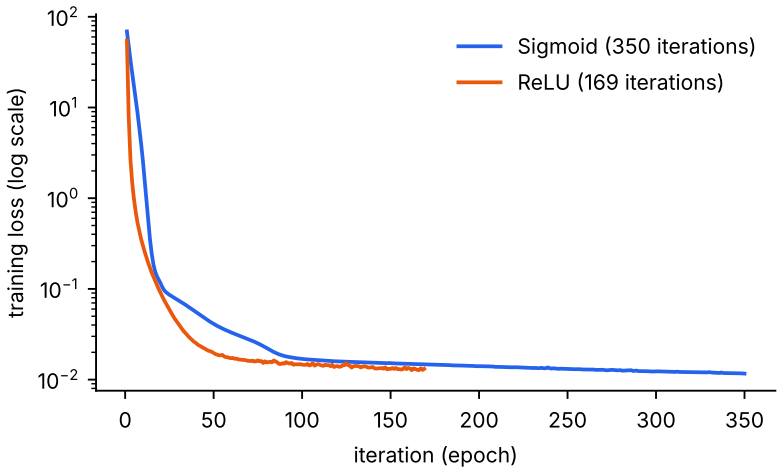


Figure A1: Training loss per iteration on a log scale, same settings as Table A4 (iteration counts match the notebook). ReLU's loss falls faster and early stopping ends it after 169 iterations; sigmoid needs 350.

Table A5: Final King County test performance (5,000 houses), predictions converted to dollars with `expm1`.

Model	Test MSE ($\$^2$)	Test RMSE ($)	Test R^2
Naive baseline (training-mean price, $538,144)	1.449×10^{11}	380,675	-0.0005
Final sigmoid MLP (64, 32), refit on all 16,613 houses, 285 iterations	$\mathbf{1.421 \times 10^{10}}$	**119,202**	**0.9019**

C. Task 3: CIFAR-10 neural-network classifier

Table A6: Architecture comparison (no regularization, 15 epochs, 45,000 train / 5,000 validation images).

Architecture	Hidden layers	Parameters	Time (s)	Best epoch	Best val. acc.	Val. loss at best
Medium	[256, 128]	820,874	188.93	14	**0.5132**	**1.3922**
Deep-Narrow	[128, 128, 128, 128]	444,170	162.78	14	0.4956	1.4153
Shallow-Wide	[512]	1,578,506	246.21	13	0.4910	1.4658
Shallow-Narrow	[128]	394,634	149.81	15	0.4836	1.4804

All: flattened 3,072 inputs in $[0, 1]$, ReLU hidden layers, Adam (lr 0.001), batch size 256, cross-entropy loss, CPU training.

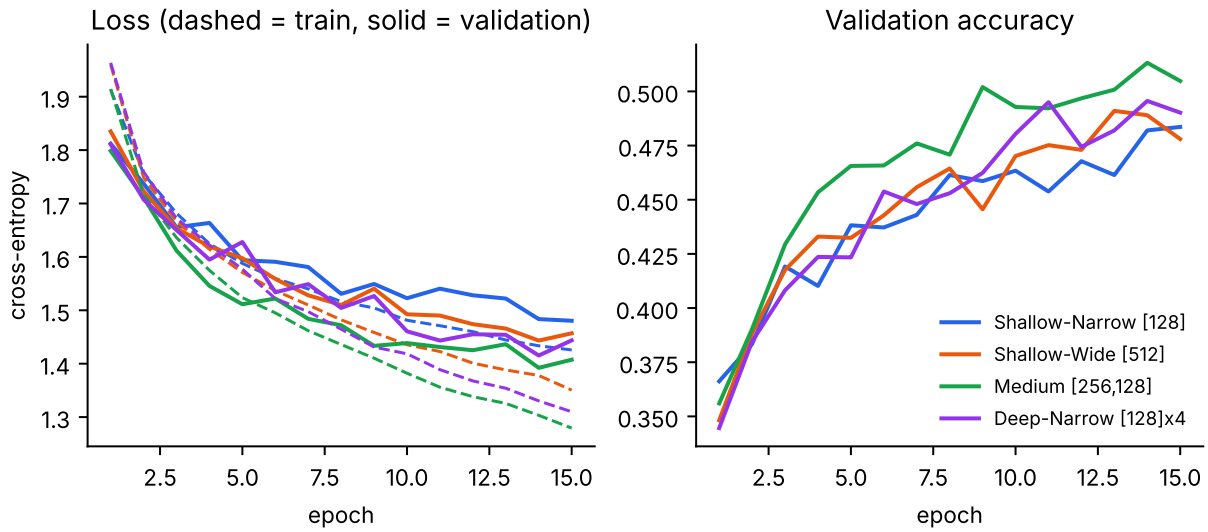


Figure A2: Per-epoch training and validation loss (left) and validation accuracy (right) for the four architectures, plotted from the epoch logs in the notebook.

Table A7: Dropout regularization on the [256, 128] architecture (15 epochs).

Dropout	Train loss (eval mode)	Val. loss	Loss gap	Best val. acc.	Time (s)
0.00	1.2619	**1.4075**	0.1455	**0.5132**	188.93
0.10	1.3155	1.4152	0.0998	0.4970	171.55
0.30	1.4775	1.5342	**0.0567**	0.4606	206.19

Train and validation losses after the final epoch, both measured with dropout disabled. Dropout reduced the gap but also reduced validation accuracy.

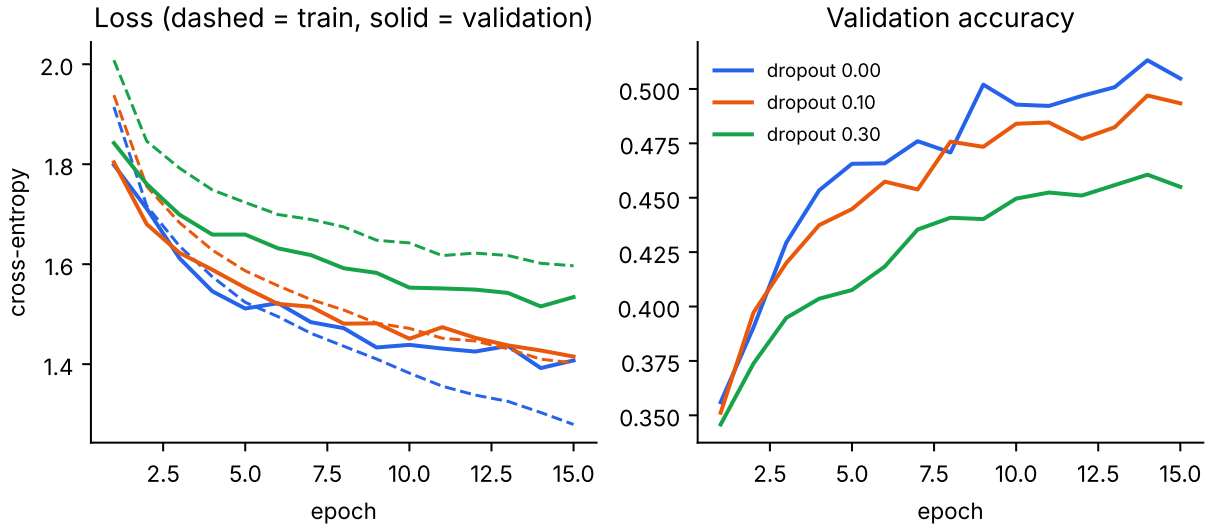


Figure A3: Per-epoch loss and validation accuracy for dropout 0, 0.10 and 0.30, plotted from the notebook's epoch logs. The training curves here include dropout's noise, which is why dropout-0.30 training loss sits above its validation loss.

Table A8: Final CIFAR-10 test-set result (10,000 images).

Final model	Training data	Epochs	Test accuracy	Test cross-entropy	Largest confusion
[256, 128], dropout 0	50,000 images	14	0.5075 (50.75%)	1.3969	cat→dog (336)

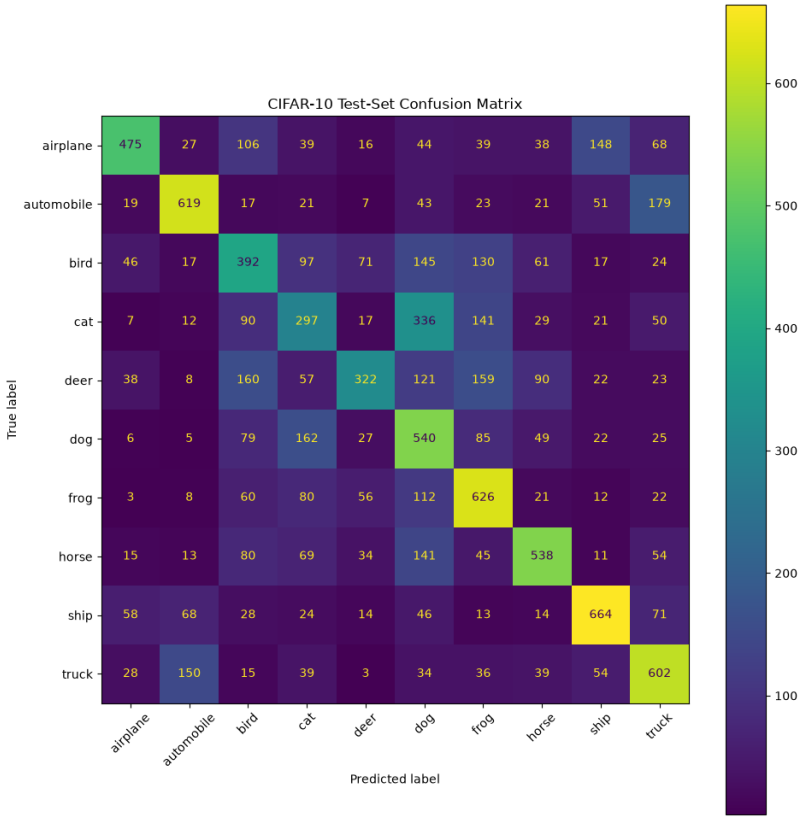


Figure A4: CIFAR-10 test-set confusion matrix for the final [256, 128] network.

Table A9: Per-class test accuracy and most frequent confusion (from Figure A4; 1,000 test images per class).

Class	Accuracy	Most often predicted as	Class	Accuracy	Most often predicted as
ship	66.4%	truck (71)	horse	53.8%	dog (141)
frog	62.6%	dog (112)	airplane	47.5%	ship (148)
automobile	61.9%	truck (179)	bird	39.2%	dog (145)
truck	60.2%	automobile (150)	deer	32.2%	bird (160)
dog	54.0%	cat (162)	cat	29.7%	dog (336)